

Fusion Graph Structure Learning-Based Multivariate Time Series Anomaly Detection With Structured Prior Knowledge

Shiming He^{1b}, Genxin Li^{1b}, Kun Xie^{1b}, *Member, IEEE*, and Pradip Kumar Sharma^{2b}, *Senior Member, IEEE*

Abstract—Multivariate time series anomaly detection (MTSAD) plays a crucial role in the Internet of Things (IoT) to identify device malfunction or system attacks. Graph neural networks (GNN) are widely applied in MTSAD to capture the spatial features among sensors. However, GNNs depend on a graph structure and explicit graph structures are not always available. To solve the problem of missing explicit graph structure, graph structure learning is introduced to learn an accurate graph structure joint with a GNNs-based anomaly detection task. However, the existing GSL-based methods provide only a partial view of the graph structure and cannot represent multiple and complex relationships. The noise of data also brings noisy edges. Therefore, we propose a fusion graph structure learning-based multivariate time-series anomaly detection with structured prior knowledge (FuGLAD). To the best of our knowledge, it appears to be the first application of fusion graphs in time series anomaly detection. FuGLAD selects three kinds of typical graph structure learners to learn as many relationship types among sensors as possible and exploits the prior similarity to evaluate the importance of all learned graphs and adaptively learn the fusion weight instead of the direct average weight. To handle noise in raw data, FuGLAD compares the neighbors of nodes by Jaccard similarity to identify and remove the noisy edges in the prior graph. Extensive experiments demonstrate that our approach outperforms state-of-the-art single-graph structure learning techniques in detection performance across four public and real-world datasets.

Index Terms—Multivariate time series, anomaly detection, graph structure learning, fusion graph.

Received 3 January 2024; revised 6 August 2024; accepted 3 September 2024. Date of publication 12 September 2024; date of current version 30 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272062 and Grant 62052501; in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139; in part by the Scientific Research Fund of Hunan Provincial Transportation Department under Grant 202143; and in part by the Open Fund of Key Laboratory of Safety Control of Bridge Engineering, Ministry of Education (Changsha University of Science and Technology), under Grant 21KB07. The associate editor coordinating the review of this article and approving it for publication was Prof. Kun Sun. (*Corresponding author: Kun Xie.*)

Shiming He and Genxin Li are with the School of Computer and Communication Engineering and Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha University of Science and Technology, Changsha 410114, China (e-mail: smhe_cs@csust.edu.cn; 22108031619@stu.csust.edu.cn).

Kun Xie is with the College of Computer Science and Electronics Engineering, Ministry of Education Key Laboratory of Fusion Computing of Supercomputing and Artificial Intelligence, Hunan University, Changsha 410082, China (e-mail: xiekun@hnu.edu.cn).

Pradip Kumar Sharma is with the Department of Computing Science, University of Aberdeen, AB24 3UE Aberdeen, U.K. (e-mail: Pradip.sharma@abdn.ac.uk).

Digital Object Identifier 10.1109/TIFS.2024.3459631

I. INTRODUCTION

IN THE Internet of Things (IoT), more and more critical infrastructures are being deployed by various sensors. The data generated by these multiple sensors is known as multivariate time series data (MTS). When abnormal changes occur in the MTS, it indicates that the equipment has failed or the system is under attack. If the anomaly is not discovered and dealt with in time, the whole system may fail and lead to economic losses [1]. Therefore, multivariate time series anomaly detection (MTSAD) stands as a crucial undertaking within the domain of IoT [2]. In the IoT, sensors rely on and interact with each other. Therefore, when conducting MTSAD, it is crucial to understand both the temporal relationships of individual indicators¹ and the interrelationships among these indicators. In recent years, graph neural networks (GNNs) have been introduced into the field of multivariate time series anomaly detection for modeling the relationships among indicators. Existing GNNs-based anomaly detection methods, such as the Arvalus and its variant D-Arvalus [3] utilize a fixed graph structure constructed from domain expert knowledge for modeling the relationships among indicators. However, explicit graph structures are not always available in every application. Therefore, how to define an accurate graph structure becomes a key task for GNNs-based anomaly detection.

As a result, graph structure learning (GSL) has arisen in MTSAD [4], [5], [6]. It learns an optimal graph structure in conjunction with the subsequent downstream anomaly detection task. Fig. 1 shows the GSL-based anomaly detection process. However, GSL-based methods still face several challenges.

- **Existing methods provide only a partial view of the graph structure, which cannot represent the multiple and complex relationships that exist between indicators.** The existing graph structure learning methods can be classified into three categories [7]: metric-based, neural network-based, and direct methods. The current GSL-based anomaly detection methods all solely adopt one of these three methods to learn a partial view of the graph structure. The ideal graph structure is not always available or may be incomplete due to the difficulty in obtaining such information or deliberate shielding to

¹Here, “indicator” signifies the time series of a particular variable.

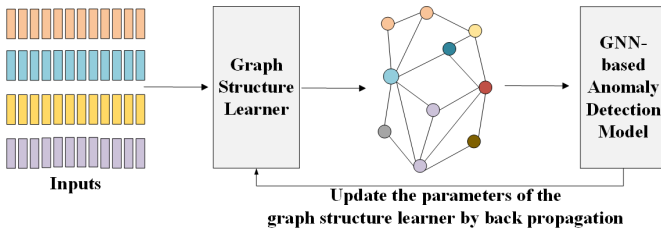


Fig. 1. A framework for anomaly detection based on graph structure learning.

protect sensitive information [8]. The learned graph structure is only an estimate of the ideal graph and only represents the relationships from a single view, which inevitably has a certain degree of information loss or misconnection [9].

i) The metric-based approach only focuses on the similarity relationship. It establishes connections between nodes with higher similarity of node embedding vector to form edges. However, the relationships between nodes still include causality and distance except for similarity. **ii) Neural network-based graph learning methods depend on the type of neural network.** If simple linear neural networks are used, learned graph structure learning may not yield satisfactory results. Conversely, complex neural network structures increase the training cost of the model.

iii) The direct approach offers great flexibility as it does not rely solely on node representation. It treats the adjacency matrix of the target graph as a free variable for learning. Nonetheless, in many practical applications, node features are equally crucial for comprehending and predicting the behavior of sensors.

- **Noise in the raw data can affect the quality of the prior graph.** To constrain the learning of graph structure, some prior information is usually added to the graph learning process. Existing methods usually utilize the original data to generate a K -nearest Neighbors (KNN) graph as prior information, termed a prior graph. However, there is usually noise in the raw data resulting in an inevitable presence of uncertain, redundant, incorrect, and incomplete connections within the prior graph, termed noisy edges. Such noisy information in the prior may damage the performance of GNNs.

To address these challenges, we propose FuGLAD, a technique that fuses multiple graph structures to refine the quality of the learning graph and uses the structured prior graph to eliminate the effects of noise in the original data. To the best of our knowledge, this is the first work to apply fusion graphs in time series anomaly detection. Our major contributions can be summarized as follows:

- To address the single view representation of individual graph structures, we propose a fusion graph structure learning method that merges various graph structures to mine multiple view relationships between sensors. It consists of two key steps: the basic graph structure learner selection and graph fusion.

- In basic graph structure learner selection, to learn as many relationship types among sensors as possible, we choose three graph structure learning methods that have been widely proven to be effective. K -Nearest Neighbour method is for the association, causal graph learning is for causality, and the fully parameterized method treats all elements in the adjacency matrix as independent variables which is the replenishment of the two former.
- In graph fusion, we propose a prior similarity-based weighted fusion method (PSF) instead of average fusion since a prior graph (such as a kNN graph derived from data) still serves as reasonable knowledge [8]. It exploits the cosine similarity between the prior graph and the learning graph to evaluate the importance of all learned graphs and adaptively learn the fusion weight, rather than simply taking the average of the learned graphs.
- To handle noise in raw data, we design a structured prior graph (SPG) generator to construct a prior graph in the structured space. SPG compares the neighbors of nodes by Jaccard similarity to identify and remove the noise edges in the prior graph.
- Extensive experiments on four public and real-world datasets are conducted to evaluate the performance of FuGLAD. Compared to the thirteen baseline methods, FuGLAD consistently has the highest F1 score.

The following sections of this paper are organized as follows: Section II reviews the related work. Section III covers preliminary knowledge and problem definition. Section V details the FuGLAD approach. Section VI discusses performance evaluation through experiments. Finally, Section VII summarizes the findings and explores future work.

II. RELATED WORK

A. Temporal Feature-Based Anomaly Detection Method

LSTM is widely used in multivariate time series anomaly detection because of its ability to process sequence data. LSTM-NDT [10] utilizes LSTM for accurate predictions while maintaining the interpretability of the whole system. A non-parametric, dynamic, and unsupervised threshold method is provided to detect anomalies. MSCRED [11] constructs a multi-scale feature matrix and uses conv-LSTM neural networks to encode the correlation between sensors and reconstruct the multi-scale feature matrix. The residual feature matrix helps to detect and diagnose anomalies. MAD-GAN [12] employs LSTM as the foundational model within the GAN framework, capturing the temporal correlation of time series, extracting potential interactions, and detecting anomalies by discrimination and reconstruction. LSTM-VAE [13] introduces the multi-modal observation and time dependence into the potential space, reconstructs the expected distribution, and evaluates the anomalies based on the reconstructed anomaly scores. OmniAnomaly [14] utilizes a stochastic recurrent neural network to capture the robust representation of normal patterns and reconstruct observations. DAGMM [15] utilizes depth self-coding and Gaussian mixture model to obtain low-dimensional representation and exploit reconstruction errors to detect anomalies. CAE-M [16] employs a

Convolutional Autoencoding Memory Network to effectively handle time series data. By utilizing a Convolutional Neural Network (CNN), CAE-M efficiently processes the input time series. The resulting output of CNN is then subjected to a bi-directional Long LSTM network, enabling the model to effectively capture and analyze long-term time trends.

Although LSTM networks have demonstrated promising results in modeling the temporal dependencies of MTS, they are unable to directly capture the correlations between different indicators [17].

B. Graph-Based Anomaly Detection Method

Recent studies have demonstrated the effectiveness of graph neural networks in predicting temporal dependencies and modeling correlations between indicators. For example, MTAD-TF [18] uses multi-scale convolution and graph attention networks to capture temporal pattern information. Similarly, MTAD-GAT [19] employs two parallel graph attention layers to model both temporal features and correlations between indicators, which are then passed to a lightweight Gated Recurrent Unit (GRU) network. Additionally, Arvalus and its variant D-Arvalus [3] represent system components as nodes and their dependencies and locations as edges to improve anomaly identification and localization.

Although graph neural network-based anomaly detection methods have effectively modeled time-series correlations between indicators, they still have some limitations. Most of the existing graph neural network-based anomaly detection methods [3], [18], [19] assume that the relationship between indicators is known or assume there is a relationship between any two indicators, which increases computational overhead with increasing dimensionality [20].

C. Graph Structure Learning-Based Anomaly Detection Method

To address MTSAD without a predefined graph structure, Graph Structure Learning-based methods have been developed. For example, GDN [4] generates an embedding vector for each node and settles to compute the similarity between the embedding vectors between any two nodes, and subsequently selects the top k nodes with the highest similarity to form the graph. This graph is then used by a graph attention network to forecast future sensor behavior. FuSAGNet [5] categorizes the sensor nodes by function and recursively encodes the nodes in each category. Subsequently, it uses the resulting embeddings to compute the similarity between the nodes and selects the top k nodes with the highest similarity to form the graph. ACGSL [21] generates dynamic graphs for each subsequence by minimizing differences between consecutive ones.

GTA [6] treats adjacency matrix elements as learnable parameters and uses a Transformer-based model to automatically learn the graph structure and temporal dependencies. GLAD [22] combines GAT and Transformer to extract global and local features for anomaly detection. MGCLAD [23] employs two directed graphs to learn intersignal and

intrasignal correlations, while GSLAD [24] uses a diffusion convolutional recurrent neural network for anomaly detection.

Although anomaly detection methods based on graph learning are able to adaptively generate a suitable graph structure for the downstream task through backpropagation, it is important to note that current approaches solely produce a partial view of the graph structure, consequently leading to the unavoidable loss of information.

III. PROBLEM DEFINITION AND PRELIMINARIES

A. Multivariate Time Series

Multivariate time series can be described as $X = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)^T \in \mathbb{R}^{K \times N}$, where K represents the number of indicators and N signifies the number of timestamps. The values of i -th indicator are represented as $\mathbf{x}^i = (x_1^i, x_2^i, \dots, x_N^i)$. The values at timestamp t are represented as $\mathbf{x}_t = (x_t^1, x_t^2, \dots, x_t^K)^T$. A history window of length ω at timestamp t is defined as the subsequence $X_t = (\mathbf{x}_{t-\omega}, \mathbf{x}_{t-\omega+1}, \dots, \mathbf{x}_{t-1})^T \in \mathbb{R}^{K \times \omega}$.

B. Graph Structure Learning in MTS

Given the time series $X \in \mathbb{R}^{K \times N}$, graph structure learning aims to construct a graph $G = (V, E)$ and its adjacency matrix $A \in \mathbb{R}^{K \times K}$. In this graph, the nodes v represent the sensors that produce the indicators, while the edges E denote the hidden relationships between these sensors. The adjacency matrix A stores the edge information, which reflects the underlying dependencies among the indicators.

C. GSL-Based Multivariate Time Series Anomaly Detection

Multivariate time series anomaly detection calculates an anomaly score for each timestamp, denoted as s_t , according to the historical subsequence X_t and the adjacency matrix A generated by GSL. An anomaly is deemed to have transpired at timestamp t if the anomaly score s_t surpasses a predetermined threshold. The process is shown as follows:

$$A = \text{GSL}(X) \quad (1)$$

$$\hat{\mathbf{x}}_t = f(X_t, A) \quad (2)$$

$$s_t = \varphi(x_t, \hat{\mathbf{x}}_t) \quad (3)$$

$$\hat{y}_t = \begin{cases} 1, & \text{if } s_t > T \\ 0, & \text{if } s_t \leq T \end{cases} \quad (4)$$

where $f()$ denotes the prediction function, $\hat{\mathbf{x}}_t$ represents the forecasted value of $\mathbf{x}_t$ at timestamp t , φ is the function used to calculate the anomaly score, and T stands for the threshold.

IV. MOTIVATION

The current graph neural network-based anomaly detection methods are all based on a partial view of the graph structure [25]. In the world, objects might form different kinds of relationships with others (e.g., sensor distance and data semantic relationship on IoT). This results in multi-view graphs that contain more than one type of edge between two nodes [26], [27]. Therefore, anomaly detection can be performed using

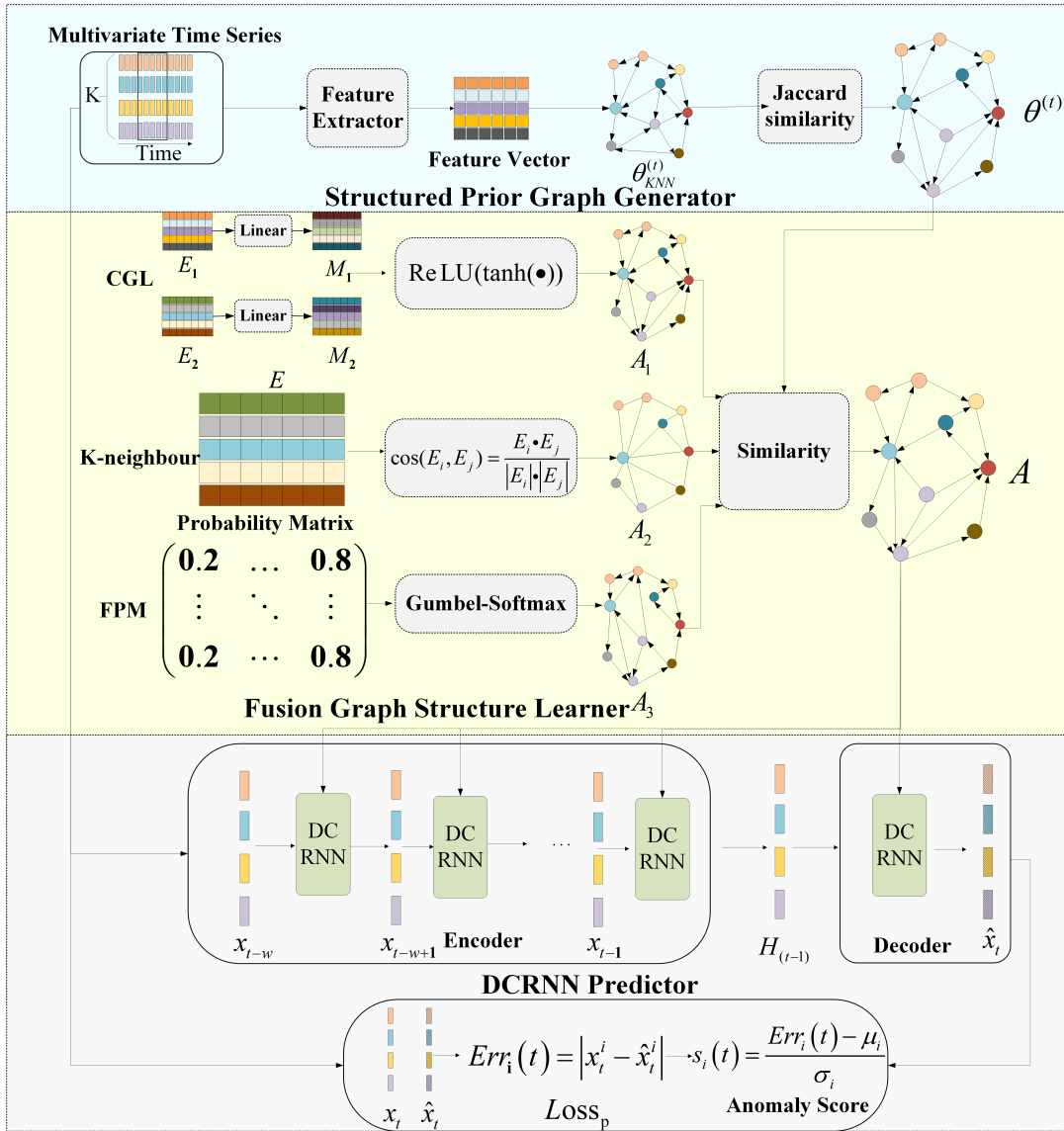


Fig. 2. The FuGLAD framework includes three primary modules. The first module, illustrated in blue, captures the relationships between indicators. The second module, in gold, generates a structured prior graph that aids the graph structure learner with prior information. The third module, in green, uses a DCRNN predictor to produce precise indicator forecasts.

a multi-view approach, where multiple views allow us to analyze the characteristics of a physical object from different perspectives. Each view can offer complementary information to the other views. For example, an anomaly may not be identifiable in one view but can be in another. There exists a plethora of research in data mining focusing on multi-view learning [28], [29]. However, multi-view learning for anomaly detection is still in its infancy. For this reason, we propose a multi-graph structure learning method for anomaly detection.

Although multiple views contain rich information, there may be inconsistencies among them. Digesting the relationship between views is crucial for the success of multi-graph learning algorithms, as two views may provide opposite/complementary information for anomaly detection [25]. To this end, we propose a fusion approach based on a priori graph. The constraints of a priori graph make the learned graphs as close as possible without generating opposite

information and not the same to provide complementary information for each other.

V. PROPOSED METHODOLOGY

A. Overview

To address the single-view representation of individual graph structures, we propose FuGLAD, which fuses multiple basic learned graph structures to accurately represent relationships. Fig. 2 shows the framework of FuGLAD. It primarily consists of three main components:

- **Structured Prior Graph Generator:** It acquires a structured prior graph from noise data that offers prior information for graph learning.
- **Fusion Graph Structure Learner:** It utilizes three mainstream graph structure learners to generate multiple graphs and fuse them to characterize complex relationships among sensors.

TABLE I
LIST OF NOTATIONS

Notation	Meaning
X	Multivariate time series
x^i	The i -th indicator values
x_t	The indicator values at timestamp t
$\hat{x}$	The prediction
K	The number of indicators
N	The number of timestamps
y_t	The label of at timestamp t
G	Graph
A	Adjacency matrix
A_i	The adjacency matrix of i -th basic graph structure
θ	Prior Adjacency matrix
$V_i^{(t)}$	The feature vector of sensor i at timestamp t
$E_i^{(t)}$	The embedding vector of sensor i at timestamp t
g_i	The samples from the Gumbel distribution
π_i	The probabilities of i -th class
τ	The temperature parameter
$H^{(t)}$	The hidden features at timestamp t
$Err_1(t)$	The prediction error at timestamp t for sensor i
$s(t)$	Anomaly scores

- **DCRNN Predictor and Anomaly Score:** It utilizes the diffusion convolutional recurrent neural network (DCRNN) for predicting the future values of each sensor and calculates the anomaly score by the difference between the prediction and ground truth.

The process of FuGLAD can be summarized as follows: Firstly, the fusion graph structure learner learns multiple basic graph structures and fuses them to obtain a fused adjacency matrix A . Secondly, based on the fused adjacency matrix A , a DCRNN predictor predicts the value $\hat{x}_t$ of the t -th timestamp from the historical observations $x_{t-w}, \dots, x_{t-1}$. Then, the basic graph structures are jointly trained with the prediction task, and the normalized prediction error between the predicted value $\hat{x}_t$ and the ground truth x_t is treated as the anomaly score. Finally, a grid search algorithm is used to select a threshold for best detection accuracy. Once the anomaly score at timestamp t exceeds the threshold, timestamp t is taken as an anomaly. The notation employed here is outlined in Table I.

B. Structured Prior Graph Generator

Using prior graphs extracted from raw data as a guide is an effective and widely applied approach to constrain the direction during the training of graph structure learning, ensuring the quality of the learned graph [8], [30]. Many prior graph generation methods extract node features from the raw data. The features are then utilized to compute a K -Nearest Neighbor (KNN) graph, which serves as the prior knowledge for the learned graph. Nevertheless, the presence of noise in the data may corrupt the node feature vectors, resulting in an inevitable presence of uncertain, redundant, incorrect, and incomplete connections within the prior graph, termed noisy edges. To reduce the noisy edges, inspired by the common-neighbor-based metric [31], we propose a structured prior graph generator based on Jaccard similarity. Jaccard similarity compares the common neighbors between nodes, enabling the consideration of a broader spatial structure.

It goes beyond the direct connection between two nodes and takes into account their position in the entire space and their relationships with other nodes. Nodes with a greater number of common neighbors display higher similarity. Consequently, the presence of a noisy edge can be alleviated by tallying the count of common neighbors, as long as this noisy edge is not shared among multiple nodes, which is typically unlikely.

We calculate the Jaccard similarity among nodes on the KNN graph and generate a structured graph by selecting the top- k nodes with the highest Jaccard similarity for each node as its neighbor. This process effectively transforms the traditional KNN graph into a structured space. In the structure space, the features have the capability to encode a greater amount of texture information by perceiving the data distribution thus being more robust [32].

Initially, we construct a feature extractor for characterizing the behavior of each sensor as follows:

$$V_i^{(t)} = \text{FC}(\text{Conv}(\text{Conv}(x_i^{(t)}))) \quad (5)$$

where $x_i^{(t)} \in R^w$ represents the subsequence for sensor i at timestamp t , $V_i^{(t)}$ denotes the feature vector of sensor i at timestamp t , Conv refers to the one-dimensional convolutional layer, and FC refers to the fully connected layer.

Subsequently, the cosine similarity is computed for the feature vectors of every node pair as follows:

$$\cos(V_i^{(t)}, V_j^{(t)}) = \frac{V_i^{(t)} \bullet V_j^{(t)}}{\|V_i^{(t)}\| \bullet \|V_j^{(t)}\|} \quad (6)$$

where $\|\cdot\|$ denotes magnitude. Afterward, the top- k nodes with the highest similarity are selected as the neighbors of N_i , noted as $v^{(t)}(N_i, k) = \{N_{i,1}, N_{i,2}, \dots, N_{i,k}\}$. It generate a KNN graph $\theta_{KNN}^{(t)}$.

Finally, we compute the Jaccard similarity of the node pair on the KNN graph to compare the common-neighbor. The k nodes with the highest Jaccard similarity are designated as the neighbors of node N_i to generate a structured graph as follows:

$$S^{Jac}(v(N_i, k), v(N_j, k)) = \frac{|v(N_i, k) \cap v(N_j, k)|}{|v(N_i, k) \cup v(N_j, k)|} \quad (7)$$

$$\theta_{ij}^{(t)} = 1, j \in \text{topk}(S^{Jac}(v(N_i, k), v(N_j, k))) \quad (8)$$

where S^{Jac} represents the Jaccard similarity, $\theta_{ij}^{(t)}$ denotes the element in the i -th row and j -th column of the prior graph, and $\text{topk}(\cdot)$ function extracts the top k values, enabling focus on the most relevant data points based on the specified criteria. [4].

We take an example to illuminate how Jaccard similarity removes the noisy edge. As shown in Fig. 3, there are nine nodes, and C_i represents the neighbor set of node i . The edge between nodes 2 and 3 is a noisy edge (marked in red). According to Eq. (7), we can obtain the Jaccard similarity of nodes 1 and 2, which is $S^{Jac}(C1, C2) = \frac{C1 \cap C2}{C1 \cup C2} = \frac{2}{2+2+3} = 0.286$. Similarly, the Jaccard similarity of node 2 and 3 is $S^{Jac}(C2, C3) = \frac{C2 \cap C3}{C2 \cup C3} = \frac{3}{1+3+2} = 0.5$. $S^{Jac}(C2, C3)$ is smaller than $S^{Jac}(C1, C2)$. When selecting the top- k neighbors, node 2 prioritizes choosing node 1 as a neighbor over

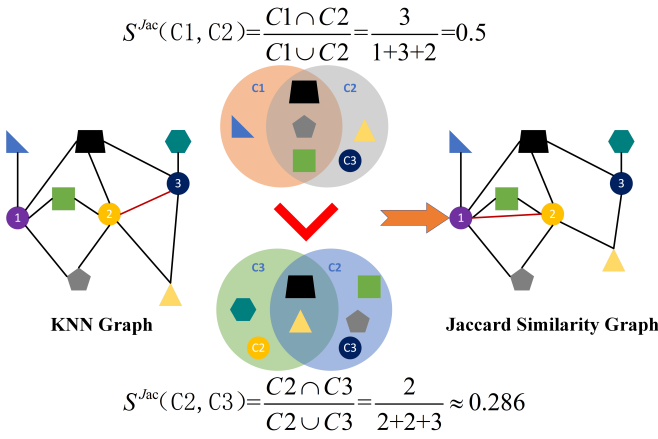


Fig. 3. A simple example of eliminating the noisy edge by the Jaccard similarity.

node 3. Consequently, by doing so, we can eliminate the noisy edge (2, 3) and obtain the correct edge (1, 2).

C. Fusion Graph Structure Learner

In graph structure learning-based anomaly detection, the ideal graph structure is not always available or may be incomplete. The currently given adjacency matrix is only an approximation of the ideal graph structure. Consequently, this approximation leads to a certain extent of information loss. To address this issue, it is advantageous to have multiple adjacency matrices that represent various viewpoints of the graph structure and fusion them for anomaly detection. Therefore, we propose fusion graph structure learning. As shown in Fig. 2, fusion graph structure learning consists of two main parts: basic graph structure learners and graph fusion.

1) *Basic Graph Structure Learner*: The existing graph structure learning methods can be classified into three categories [7]: metric-based, neural network-based, and direct methods. From these three types of methods, we select the three most representative graph structure learners to individually characterize the relationships between nodes from different perspectives: K -Nearest neighbor, causal graph learning, and fully parameterized method. K -Nearest neighbor method is for the association, causal graph learning is for causality, and the fully parameterized method treats all elements in the adjacency matrix as independent variables which is the replenishment of the two formers.

a) *K neighbor Method (K neighbor)*: Among the metric-based graph learning methods, we select the method based on pairwise cosine similarity [4] for the association. It defines a node embedding for each node and the top- k highest cosine similarity nodes are treated as neighbors, which generates the first adjacency matrix A_1 . It is referred to as the K neighbor, whose procedure is shown as follows:

$$\cos(E_i^{(t)}, E_j^{(t)}) = \frac{E_i^{(t)} \bullet E_j^{(t)}}{\|E_i^{(t)}\| \bullet \|E_j^{(t)}\|} \quad (9)$$

$$A_1^{(i,j)} = 1, j \in \text{topk}(\cos(E_i^{(t)}, E_j^{(t)})) \quad (10)$$

where $\|\cdot\|$ denotes the magnitude, and $\text{topk}(\cdot)$ extracts the top k values, enabling focus on the most relevant data points based on the specified criteria [4].

b) *Causal graph learning (CGL) method*: Among the neural network-based graph learning methods, we select a unidirectional graph structure learning method that can model causal relationships between sensors [33]. It defines a source and a destination node embedding for each node and generates the second adjacency matrix A_2 . It is referred to as CGL, whose procedure is shown as follows:

$$M_1 = \tanh(\alpha E_1 \beta_1) \quad (11)$$

$$M_2 = \tanh(\alpha E_2 \beta_2) \quad (12)$$

$$A_2 = \text{ReLU}(\tanh(\alpha(M_1 M_2^T - M_2 M_1^T))) \quad (13)$$

for $i = 1, 2, \dots, K$:

$$\begin{aligned} idx &= \arg \text{topk}(A_2^{(i,:)}) \\ A_2^{(i,j)} &= 1, \quad j \in idx \end{aligned} \quad (14)$$

where E_1 and E_2 denote the source and destination node embeddings, β_1, β_2 are model parameters, α represents the activation function's saturation rate, and $\arg \text{topk}(\cdot)$ returns the indices of the top- k largest values in a vector.

To reduce the training cost, the CGL method only employs a single-layer network, while it can be complemented by the other two graph learning approaches to ensure performance.

c) *Fully Parameterized Method (FPM)*: Among the direct methods, we select one that employs the Gumbel-softmax technique to construct the graph structure [6]. It exploits a probability matrix to generate the third adjacency matrix A_3 . It is referred to as FPM, whose procedure is shown as follows:

$$g_i = -\log(-\log(u)), u \sim \text{Uniform}(0, 1) \quad (15)$$

$$z_1^{i,j} = \frac{\exp\left(\left(\log \pi_1^{i,j} + g^{i,j}\right)/\tau\right)}{\sum_{v \in \{0,1\}} \exp\left(\left(\log \pi_v^{i,j} + g^{i,j}\right)/\tau\right)} \quad (16)$$

where u represents samples drawn from the $\text{Uniform}(0,1)$ distribution, $g^{i,j}$ denotes the Gumbel distribution, and $\pi_1^{i,j}$ is the value in row i and column j of the probability matrix $\pi_1 \in \mathbb{R}^{K \times K}$, indicating the probability that node i is connected to node j in the graph. The temperature parameter τ controls the distribution, with $z_1^{i,j}$ approaching 0 or 1 as τ approaches 0, making the Gumbel-Softmax distribution converge to the class distribution. Finally, the element in row i and column j of the third adjacency matrix $A_3^{(i,j)}$ is set to $z_1^{i,j}$, which is 1 with probability $\pi_1^{i,j}$.

2) *Fusion Graph*: After generating these three graph structures, the most important step is how to fuse them. The simplest approach is fusing them by average. However, it ignores the importance and qualities of all learned graph structures. Therefore, we propose the prior similarity-based weighted fusion method to adaptively learn the fusion weight instead of the direct average weight.

We obtain a quantitative assessment of the similarity between each learned graph and the prior structured graph to evaluate the importance. Subsequently, we normalize the

importance by softmax to derive probability weights for learned graphs. Finally, the probability weights are used to compute a weighted sum of multiple learned graphs to generate a fusion adjacency matrix to characterize the relationship between sensors.

$$\alpha_i = \text{Softmax}(\cos(A_i, \theta^{(t)})) \quad (17)$$

$$A = \sum_{i=1}^3 \alpha_i A_i \quad (18)$$

where $\cos(A, B)$ denotes the cosine similarity of A and B .

D. DCRNN Predictor

DCRNN [34] is an exceptional spatio-temporal graph convolutional networks, which can capture the temporal and spatial features simultaneously. It captures spatial dependencies through diffusion convolution and temporal dependencies with an encoder-decoder architecture. In the diffusion convolution operation, each node aggregates information from its neighbors up to L steps away, where L is referred as the diffusion degree. This operation can be performed in both the outward and inward directions, meaning that a node can gather information from its outgoing neighbors (i.e., nodes it points to) and its incoming neighbors (i.e., nodes that point to it). Therefore, we utilize DCRNN instead of GCN. The formula for DCRNN is as follows:

$$R_t = \text{sigmoid}(W_R \circ [x_t || H_{(t-1)}] + b_R) \quad (19)$$

$$C_t = \tanh(W_C \circ [x_t || (R_t \odot H_{(t-1)})] + b_C) \quad (20)$$

$$U_t = \text{sigmoid}(W_U \circ [x_t || H_{(t-1)}] + b_U) \quad (21)$$

$$H_{(t)} = U_t \odot H_{(t-1)} + (1 - U_t) \odot C_t \quad (22)$$

$$W_Q \circ Y = \sum_{l=0}^L \left(w_{l,1}^Q (D_O^{-1} A)^l + w_{l,2}^Q (D_I^{-1} A^T)^l \right) Y \quad (23)$$

where D_O and D_I are the out-degree matrix and in-degree matrix, $||$ is concatenation operation, R_t is the reset gate, C_t is the candidate hidden state, U_t is the update gate, $w_{l,1}^Q$, $w_{l,2}^Q$, b_Q are the model parameter, and the diffusion degree L is a hyperparameter.

The DCRNN-based predictor works as follows. In the encoder, $x_{t'}$ and node feature $H_{(t'-1)}$ enter diffusion convolutional lay to get node feature $H_{(t')}$ at the next timestamp t' . The encoder updates node feature $H_{(t)}$ from timestamp $t - w$ to timestamp $t - 1$ and obtains the total node feature $H_{(t-1)}$ of this subsequence. In the decoder, the total node feature $H_{(t-1)}$ is the input. After a layer of the decoder model, a hidden feature and predicted value $\hat{x}_t$ at timestamp t are obtained, as shown in Fig. 2.

E. Loss Function

Firstly, we use the mean absolute error (MAE) $loss_p$ between the prediction and ground truth as the loss function for the prediction task.

$$loss_p = \frac{1}{K} \sum_{i=1}^K |\hat{x}_t^i - x_t^i| \quad (24)$$

where $\hat{x}_t^i$ and x_t^i represent the predicted value and ground truth of the i -th indicator at timestamp t , respectively.

Secondly, We utilize the cross-entropy between the prior graph structure $\theta^{(t)}$ and the fusion graph structure A as graph learning loss to constraints the learning graph during model training.

$$loss_g = \sum_{ij} -\theta_{i,j}^{(t)} \log A_{i,j} - (1 - \theta_{i,j}^{(t)}) \log(1 - A_{i,j}) \quad (25)$$

Our aim is to optimize the model's performance in accurately predicting the indicators and capturing the underlying relationships among them. We achieve this by balancing the MAE loss function for the prediction task and the cross-entropy loss function for the graph learning task. To mitigate over-fitting, an L_2 regularization term is incorporated into the loss function. The total loss function of the model is as follows:

$$loss = loss_p + \lambda_1 loss_g + \lambda_2 \|\mathbf{w}\|_2^2 \quad (26)$$

where the parameters λ_1 and λ_2 is the regularization magnitude.

F. Anomaly Score and Threshold Selection

The purpose of anomaly detection is to identify the abnormal situation that deviates from the normal behavior according to the ground truth and prediction. Therefore, we first compute the prediction error $Err_i(t)$ of the sensor i at timestamp t by calculating the difference between ground truth and the prediction.

$$Err_i(t) = |x_t^i - \hat{x}_t^i| \quad (27)$$

Secondly, we normalize the prediction error and compute the anomaly score at timestamp t by selecting the highest value among all sensors.

$$s(t) = \max_i s_i(t) = \max_i \frac{Err_i(t) - \mu_i}{\sigma_i} \quad (28)$$

where μ_i and σ_i are the mean and standard deviation of $Err_i(t)$, respectively.

Grid search is crucial for finding the optimal threshold. It involves defining the threshold bounds as the maximum and minimum values of $s(t)$. We exhaustively search all possible thresholds with a step size of 0.01, selecting the threshold that yields the highest F1 score. Additionally, we apply a point adjustment strategy for anomaly scores.

VI. EXPERIMENTS EVALUATION

We thoroughly describe our experiments and address the following research questions:

- **RQ1 (Effectiveness and Efficiency):** Does FuGLAD exceed the baseline methods in both effectiveness and efficiency?
- **RQ2 (Parameter Sensitivity):** How sensitive is FuGLAD to various hyperparameters?
- **RQ3 (Structured Prior Graph Performance):** Whether the structured prior graph can effectively handle data with noise?

TABLE II
DETAILED INFORMATION OF DATASETS

Dataset	SWAT	WADI	MSL	SMAP
Dimension	51	127	55	25
# of training	495000	762970	58317	135183
# of testing	450000	172800	73729	427617
Anomalies	11.97%	5.99%	10.72%	13.13%

- **RQ4 (Fusion Graph Structure Learning Performance):** Does our proposed fusion graph structure learning method outperform other single graph structure learning methods and other fusion methods?

A. Datasets

We use four real-world public datasets, whose detailed information is shown in Table II.

- **Safe Water Treatment (SWAT) Dataset:** Collected by the Singapore Public Utilities Bureau, this dataset spans 11 days with continuous 24-hour monitoring. It includes network traffic and data from 51 sensors and actuators on a water treatment test bench.
- **Water Distribution (WADI) Dataset:** An extension of the SWAT platform, WADI offers a more comprehensive view of water treatment, storage, and distribution. It covers 16 days (14 for routine operations and 2 for attack simulations) using 127 sensors and actuators.
- **Mars Science Laboratory Rover (MSL) Dataset:** This dataset from NASA features 55 indicators across 27 unique entities related to the Mars rover's sensors and actuators.
- **Soil Moisture Active Passive (SMAP) Dataset:** Collected by NASA using the Mars probe, this dataset includes 25 indicators for 55 soil samples and telemetry data.

In order to handle voluminous raw data, the SWAT and WADI datasets undergo a down-sampling process every 10 seconds, capturing the median. During this period, once an anomaly occurs, it is tagged as an anomaly.

B. Evaluation Metrics

We utilize F1-Score (F1), precision (Pre), and recall (Rec) to evaluate the effectiveness of anomaly detection.

$$\text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (29)$$

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (30)$$

$$F1 = \frac{2 \times \text{Prec} \times \text{Rec}}{\text{Prec} + \text{Rec}}. \quad (31)$$

where TP, TN, FP, and FN represent the numbers of true positives, true negatives, false positives, and false negatives, respectively.

Additionally, the efficiencies of training and inference can be evaluated by metrics such as training time per epoch, inference time, memory usage, and the number of parameters.

C. Baselines

The baselines are the following thirteen methods.

- **AE:** The input data is reconstructed by an autoencoder, and the resulting reconstruction error is used as the anomaly score.
- **IF [35]:** The Isolation Forest algorithm identifies anomalies by isolating data points in a tree-based model, effectively detecting outliers.
- **DAGMM [15]:** This method combines deep autoencoders with Gaussian Mixture Models to model normal data distribution and identify anomalies through reconstruction errors.
- **LSTM-NDT [10]:** This method utilizes LSTM networks to capture contextual information and dependencies in sequential data for anomaly detection.
- **LSTM-VAE [13]:** This method projects multimodal observations and time dependencies into a latent space, using a VAE based on LSTM to reconstruct the expected distribution.
- **MAD-GAN [12]:** This method uses LSTM within a GAN framework to model temporal correlations in time series data.
- **OmniAnomaly [14]:** This method is a priori-driven stochastic model used to detect timestamp anomalies, returning the reconstruction probability directly.
- **USAD [36]:** This method trains an encoder-decoder framework in an adversarial manner to achieve fast and efficient training.
- **AT-DCAEP [37]:** This method integrates a convolutional autoencoder for feature extraction and an attention-based prediction network to capture temporal dependencies, which operates in an unsupervised manner, eliminating the need for extensive data labeling.
- **MTAD-GAT [19]:** This method considers the relationships between sensors as a complete graph and uses a graph attention neural network for anomaly detection.
- **GDN [4]:** This method builds a graph structure based on pairwise cosine similarity between nodes, uses attention-based graph neural networks to learn time series dependencies, and predicts future behaviors.
- **FuSAGNet [5]:** The graph structure is learned by pairwise cosine similarity between recursive sensor embeddings. Subsequently, it employs a sparse autoencoder to derive a sparse representation of the input data, which is then inputted into a graph attention network for forecasting.
- **GTA [6]:** This method automatically learns the graph structure by a direct approach and leverages graph convolution along with a transformer-based architecture to capture temporal dependencies.

D. Settings

The experimental parameters are listed in Table III. The experiments were run with Python 3.8, PyTorch 1.10, and CUDA 11.3, on a server featuring an Intel(R) Xeon(R) Platinum 8255C CPU and an NVIDIA RTX 3090 GPU. In the case of the baseline methods, we utilized the default

TABLE III
EXPERIMENTAL PARAMETER SETTING

Parameter	window size ω	regularization parameter λ_1	regularization parameter λ_2	learning rate	batchsize	epochs
MSL	12	1	0.0001	0.005	64	40
SMAP	12	1	0.0002	0.005	64	40
SWAT	15	1	0.0007	0.005	64	40
WADI	15	1	0.0006	0.005	64	40

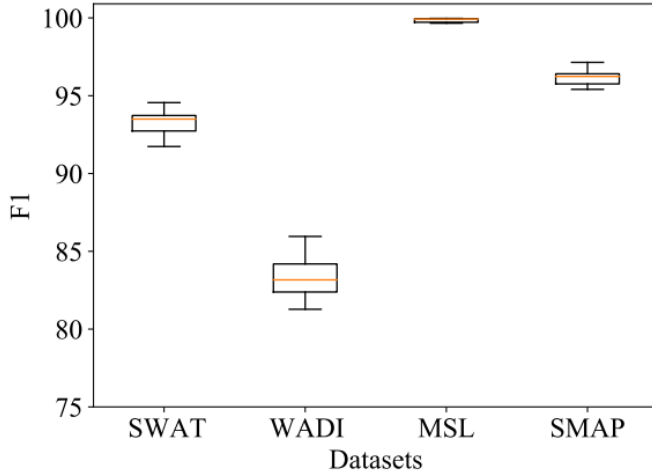


Fig. 4. Distributions of the F1 scores on the four datasets.

experimental parameters as outlined in the paper. Specifically, the window size ω for GDN and FuSAGNet is set to 5, while for GTA, it is set to 60.

E. RQ1. Effectiveness and Efficiency

Initially, a performance comparison is made between FuGLAD and all baselines. The recorded outcomes are presented in Table IV. Moreover, to demonstrate the stability of our model, we conduct several experiments and plot box plots of the model's F1 scores on four datasets, which are shown in Fig. 4. Finally, a comparison between existing graph structure learning-based anomaly detection methods [4], [5], [6] and our method is conducted in regard to efficiency. The findings of this analysis are displayed in Table V and Table VI.

1) *Effectiveness*: FuGLAD demonstrates significant superiority over other baseline methods. Compared to the most recent GSL-based anomaly detection methods, such as GDN [4], FuSAGNet [5] and GTA [6], FuGLAD enhances learning graph quality through the fusion graph structure and structured prior graph. In addition, as shown in Fig. 4, FuGLAD has stable performance.

All methods perform worse on the WADI dataset compared to others due to its lower anomaly rates, as shown in Table II. However, FuGLAD still shows the best performance on WADI over the baseline methods. This demonstrates FuGLAD's effectiveness in handling high dimensional time series data and imbalanced samples, rendering it well-suited for practical applications.

FuSAGNet's subpar anomaly detection performance on the SMAP and MSL datasets can be attributed to its design focus

on Cyber-Physical Systems (CPSs), which classifies sensors by their specific process functions [5]. FuSAGNet incorporates sensors tailored to these specific processes, whereas the sensors in the SMAP and MSL datasets do not have a clear functional classification, leading to reduced effectiveness.

2) *Efficiency*: We compare our method with the existing GSL-based anomaly detection methods in terms of training time on four datasets and inference efficiency on the SWAT dataset. The results are shown in Table V and Table VI.

Compared to FuSAGNet [5] and GTA [6], FuGLAD has shorter training time and inference time. The training time of GDN [4] is shorter than that of FuGLAD, but the performance of GTA is much worse than that of FuGLAD. GDN's shorter training time and inference time can be attributed to its simplistic architecture, which only uses a graph attention network. In contrast, GTA has the longest training time and inference time because it uses Transformer, increasing its complexity. FuSAGNet, which employs a sparse encoder SAE and a graph attention network, requires joint training of reconstruction and prediction models, resulting in performance and time that fall between those of GDN and GTA. FuGLAD, with DCRNN for prediction, significantly reduces time while maintaining model performance. When considering memory usage and the number of parameters, GDN and FuSAGNet exhibit lower memory usage and parameters than the other two methods. FuGLAD demonstrates lower memory usage and parameters compared to GTA, attributed to FuGLAD's efficient network structure.

F. RQ2. Parameter Sensitivity

To show the robustness of our approach to varying parameters, we examine how different hyperparameters affect the model's performance.

1) *Window Size*: In this experiment, we examine the impact of window size on our model's effectiveness and training time. The performance of the FuGLAD model with various window sizes can be observed in Fig. 5. Additionally, Table VII illustrates the training time per epoch of FuGLAD, also with different window sizes. The experimental findings indicate that the window size has an impact on both the anomaly detection performance and training time. In situations where a smaller window is utilized, FuGLAD can detect anomalies more swiftly due to the shorter training time for smaller inputs. However, if the window size is excessively large, shorter anomalies may be concealed among a substantial number of data points within the window, resulting in a decrease in the model's effectiveness. To strike a reasonable balance between the F1 score and training time, a window size of 15 is

TABLE IV

PRECISION, RECALL AND F1 SCORE ON THE FOUR DATASETS. THE HIGHEST AND SECOND-HIGHEST RESULTS ARE HIGHLIGHTED WITH BOLDFACE AND UNDERLINE, RESPECTIVELY

Method	Pre	SWAT Rec	F1	Pre	WADI Rec	F1	Pre	SMAP Rec	F1	Pre	MSL Rec	F1
AE	72.6	52.6	61.1	34.3	34.3	34.3	72.1	97.9	77.7	85.3	97.4	87.9
IF	96.2	73.2	83.1	62.4	61.6	62.0	44.2	51.1	46.7	56.8	67.4	59.8
DAGMM	27.5	69.5	39.4	54.4	27.0	36.1	63.3	99.8	71.2	75.6	98.0	81.1
LSTM-NDT	77.8	51.1	61.7	1.4	78.2	2.7	85.2	73.3	78.8	62.9	<u>100.0</u>	77.2
LSTM-VAE	96.2	59.9	73.9	<u>87.8</u>	14.5	24.8	71.6	98.8	75.6	86.0	97.6	85.4
MAD-GAN	99.0	63.7	77.5	41.4	33.9	37.3	81.6	92.2	86.5	85.2	99.3	91.7
OmniAnomaly	72.2	98.3	83.3	26.5	98.0	41.7	75.9	97.6	85.4	91.4	88.9	90.1
USAD	100.0	56.0	71.8	43.1	22.5	29.6	74.8	96.3	84.2	79.5	99.1	88.2
AT-DCAEP	99.7	67.7	80.7	78.8	82.9	80.8	89.0	99.9	94.2	95.2	99.9	97.5
MTAD-GAT	21.0	64.5	31.7	11.7	30.6	16.9	79.9	<u>99.9</u>	88.8	79.2	98.2	87.7
GDN	99.4	68.1	80.8	97.5	40.2	56.9	74.8	98.9	85.2	93.1	98.9	95.9
FuSAGNet	98.7	72.6	83.7	82.9	47.8	60.7	9.3	67.9	16.5	80.6	98.9	88.9
GTA	93.9	85.7	<u>89.6</u>	79.6	79.4	79.5	<u>89.1</u>	91.8	90.4	91.0	91.2	91.1
FuGLAD	97.59	<u>91.69</u>	94.55	86.47	<u>85.44</u>	85.95	94.44	100	97.14	99.93	100	99.96
Percentage Increase	+3.93%	+6.99%	+5.52%	+9.73%	+3.06%	+6.37%	+6.11%	+0.10%	+3.12%	+4.97%	+0.10%	+2.52%

TABLE V

TRAINING TIME PER EPOCH(S)

Method	SWAT	WADI	SMAP	MSL
GDN	11.13	42.4	0.67	0.57
FuSAGNet	44.23	75.29	1.57	1.14
GTA	107.38	154.33	8.46	4.17
FuGLAD	27.13	45.03	1.43	0.78

TABLE VII

TRAINING TIME PER EPOCH(S) WITH DIFFERENT WINDOW SIZE

ω	5	12	15	30	60
MSL	0.73	0.78	0.93	1.25	2.95
SMAP	1.3	1.43	2.0	2.53	5.95
SWAT	21.23	24.45	27.13	37.28	86.93
WADI	34.45	41.6	45.03	59.68	131.2

TABLE VI

INFERENCE EFFICIENCY OF DIFFERENT GSL-BASED METHODS

Method	Inference Time (S)	Memory Usage (GiB)	Parameters
GDN	7.71	5.14	4419
FuSAGNet	26.07	5.65	608375
GTA	29.07	8.84	2160315
FuGLAD	19.37	7.82	1586341

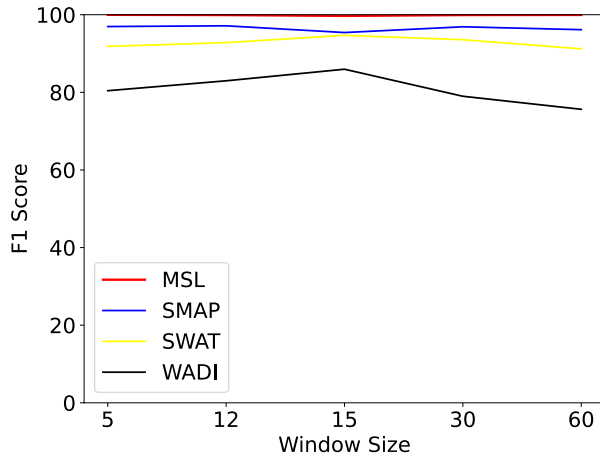


Fig. 5. The impact of window size.

employed for the SWAT and WADI datasets, while a window size of 12 is utilized for the MSL and SMAP datasets.

2) *Regularization Parameter*: In this subsection, we present the impact of regularization parameters on the anomaly detection results across four datasets. In this experiment, we vary

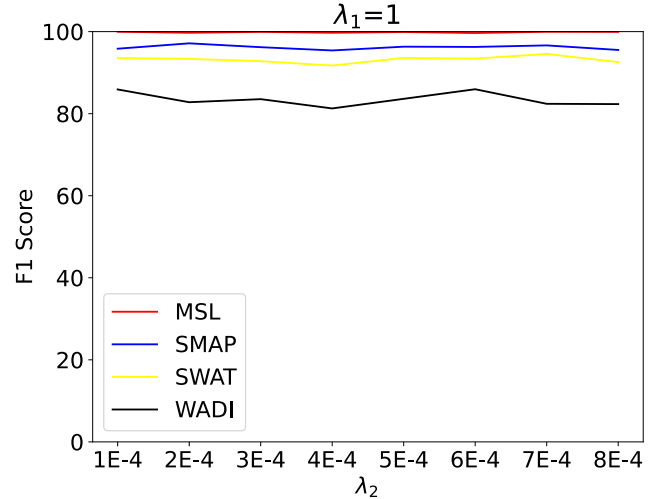


Fig. 6. The impact of regularization parameter.

λ_2 from 0.0001 to 0.0008. Based on the results shown in Fig. 6, FuGLAD demonstrates that the model has stable performance at different regulation parameters.

G. RQ3. Structured Prior Graph Performance

To demonstrate the effectiveness of the structured prior graph in mitigating the impact of noise in data, we conduct some experiments. We randomly generate noise from a normal distribution $Z \sim \mathcal{N}(0, 0.2)$ and add it to the raw data. Subsequently, we compare the anomaly detection performance of models using KNN prior graph $\theta_{KNN}^{(i)}$ and structured prior

TABLE VIII
MODEL PERFORMANCE WITH DIFFERENT PRIOR GRAPHS AND DATA

Noise	Method	Pre	SWAT Rec	F1	Pre	WADI Rec	F1	Pre	SMAP Rec	F1	Pre	MSL Rec	F1
✓	KNN	94.65	87.38	90.87	84.29	78.11	81.08	92.00	100.0	95.83	99.75	100.0	99.87
✓	SPG	97.02	86.58	91.50	81.78	84.45	83.09	93.38	100.0	96.57	99.93	100.0	99.96
×	KNN	95.39	91.17	93.23	83.36	83.86	83.61	91.89	100.0	95.77	99.38	100.0	99.69
×	SPG	97.59	91.69	94.55	86.47	85.44	85.95	94.44	100.0	97.14	99.93	100.0	99.96

TABLE IX
MODEL PERFORMANCE WITH DIFFERENT GRAPH STRUCTURE LEARNERS

Method	Pre	SWAT Rec	F1	Pre	WADI Rec	F1	Pre	SMAP Rec	F1	Pre	MSL Rec	F1
ω/o GSL	95.80	90.60	<u>93.13</u>	59.88	91.18	72.29	91.77	100	95.77	99.69	100	99.84
K neighbour	93.70	88.09	90.81	80.23	78.12	79.12	91.21	100	95.40	99.81	100	99.90
CGL	94.13	91.24	92.67	86.13	78.11	<u>81.93</u>	90.55	100	95.04	99.44	100	99.72
FPM	93.63	91.33	92.47	85.08	76.23	80.42	92.01	100	95.83	99.32	100	99.65
PSF	97.59	91.69	94.55	86.47	85.44	85.95	94.44	100	97.14	99.93	100	99.96

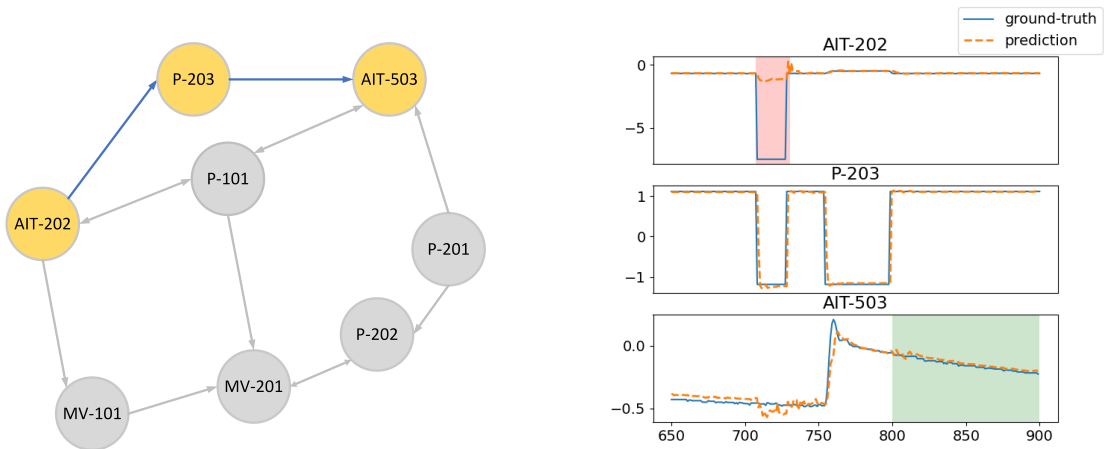


Fig. 7. **Left:** Partial graph structure learned on SWAT dataset. **Right:** Understand the relationship between sensors with ground truth and prediction.

TABLE X
INFERENCE EFFICIENCY WITH DIFFERENT GRAPH STRUCTURE LEARNERS

Method	Inference Time (S)	Memory Usage (GiB)	Parameters
ω/o GSL	21.78	10.54	1625087
K neighbour	24.45	11.20	1626079
CGL	26.31	11.54	1627065
FPM	28.26	11.62	1627814
FuGLAD	38.04	11.84	1628047

graph $\theta^{(t)}$. This comparison was done with both the raw data and the noisy data.

The experimental results are depicted in Table VIII. When noisy data is utilized, FuGLAD demonstrates a noteworthy improvement compared to the model that relies on the KNN prior graph. This is due to the effective avoidance of data noise when the prior graph is mapped into the structured space. Furthermore, even when raw data is employed, the structured prior continues to outperform the KNN prior graph. This superiority is especially pronounced when considering the SWAT and WADI datasets, as the collection process of raw

data inherently introduces noise. Additionally, the larger the dataset, the greater the amount of noise it contains.

H. RQ4. Fusion Graph Structure Learner Performance

To validate the effectiveness of fusion graphs, we compare this method with the three single graph learning methods in Section V, as well as a method that directly trains DCRNN with the prior graph from original data without graph learning (ω/o GSL). For the three single graph learning methods, we replace the fusion graph learning by K neighbor, CGL, or FPM, while the rest modules are the same as FuGLAD. ω/o GSL takes the prior graph as the graph structure of DCRNN without graph learning. As shown in Table IX, FuGLAD with multiple graphs outperforms other single graph learning methods and prior graphs on all four datasets. Moreover, we evaluate the model's inference efficiency, including inference time, memory usage, and parameters. As shown in Table X, FuGLAD requires slightly more inference time, memory, and parameters compared with the other four methods.

Furthermore, to demonstrate the effectiveness of our proposed prior similarity-based weighted fusion method,

TABLE XI
MODEL PERFORMANCE WITH DIFFERENT FUSION METHODS

Method	SWAT			WADI			SMAP			MSL		
	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1
Random SG	92.90	95.24	94.06	85.28	79.11	82.12	91.21	100	95.29	99.41	100	99.70
AVG	94.98	92.02	93.48	87.18	78.11	82.40	92.91	100	96.32	99.81	100	99.90
PSF	97.59	91.69	94.55	86.47	85.44	85.95	94.44	100	97.14	99.93	100	99.96

we conduct a comparative analysis with two other fusion approaches. The first approach stochastically selects one of the three graphs for model training (Random SG), while the second approach fuses the three graphs by averaging them (AVG). As shown in Table XI, PSF outperforms other fusion methods on all four datasets, validating the effectiveness of our proposed method.

We conduct a case study, as depicted in Fig. 7. Fig. 7 (left) illustrates a partial graph structure learned by FuGLAD, while Fig. 7 (right) presents the predictions of our model for relevant sensors. In this scenario, sensor *AIT* – 202 was compromised between timestamps 710 to 730. FuGLAD detected the attack by identifying a significant difference between the predictions and the ground truth for *AIT* – 202 during this time period. Owing to the correlation between sensors in the water treatment process, the attack on *AIT* – 202 led to the shutdown of the dosing pump *P* – 203 and affected the permeate conductivity analyzer *AIT* – 503. FuGLAD accurately predicted the changes in *P*–203 from timestamps 710 to 730 and in *AIT* – 503 from timestamps 800 to 900. The values of *P* – 203 and *AIT* – 503 followed the changes in the ground truth and were not anomalies, as demonstrated in Fig. 7 (right). This success is attributed to FuGLAD’s ability to correctly learn the correlation among the three sensors, as illustrated in Fig. 7 (left).

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose FuGLAD, a technique that fuses graph structures to refine the quality of the learning graph and exploits the structured prior graph to eliminate the effects of noise in the data. Besides, we combine the learned graph with the DCRNN predictor to efficiently forecast future sensor behavior and detect anomalies based on prediction errors. Our method outperforms the baselines on four public datasets, delivering the best performance with short-term data and reduced training overhead. Future work will focus on improving the model’s scalability for large graphs.

REFERENCES

- [1] S. He et al., “A joint matrix factorization and clustering scheme for irregular time series data,” *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119220.
- [2] S. He, Z. Li, J. Wang, and N. N. Xiong, “Intelligent detection for key performance indicators in industrial-based cyber-physical systems,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5799–5809, Aug. 2021.
- [3] D. Scheinert, A. Acker, L. Thamsen, M. K. Geldenhuys, and O. Kao, “Learning dependencies in distributed cloud applications to identify and localize anomalies,” in *Proc. IEEE/ACM Int. Workshop Cloud Intell.*, May 2021, pp. 7–12.
- [4] A. Deng and B. Hooi, “Graph neural network-based anomaly detection in multivariate time series,” in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
- [5] S. Han and S. S. Woo, “Learning sparse latent graph representations for anomaly detection in multivariate time series,” in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2977–2986.
- [6] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, “Learning graph structures with transformer for multivariate time-series anomaly detection in IoT,” *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [7] Y. Zhu et al., “A survey on graph structure learning: Progress and opportunities,” 2021, *arXiv:2103.03036*.
- [8] C. Shang, J. Chen, and J. Bi, “Discrete graph structure learning for forecasting multiple time series,” 2021, *arXiv:2101.06861*.
- [9] K. Yao, J. Liang, J. Liang, M. Li, and F. Cao, “Multi-view graph convolutional networks with attention mechanism,” *Artif. Intell.*, vol. 307, Jun. 2022, Art. no. 103708.
- [10] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, “Detecting spacecraft anomalies using LSTMs and non-parametric dynamic thresholding,” in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 387–395.
- [11] C. Zhang et al., “A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1409–1416.
- [12] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, “MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks,” in *Proc. Int. Conf. Artif. Neural Netw. (ICANN)*. Munich, Germany: Springer, 2019, pp. 703–716.
- [13] D. Park, Y. Hoshi, and C. C. Kemp, “A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [14] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust anomaly detection for multivariate time series through stochastic recurrent neural network,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 2828–2837.
- [15] B. Zong et al., “Deep autoencoding Gaussian mixture model for unsupervised anomaly detection,” in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–19.
- [16] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, “Unsupervised deep anomaly detection for multi-sensor time-series signals,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
- [17] S. Tuli, G. Casale, and N. R. Jennings, “TranAD: Deep transformer networks for anomaly detection in multivariate time series data,” 2022, *arXiv:2201.07284*.
- [18] Q. He, Y. J. Zheng, C. L. Zhang, and H. Y. Wang, “MTAD-TF: Multivariate time series anomaly detection using the combination of temporal pattern and feature pattern,” *Complexity*, vol. 2020, pp. 1–9, Oct. 2020.
- [19] H. Zhao et al., “Multivariate time-series anomaly detection via graph attention network,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.
- [20] W. Zhang, C. Zhang, and F. Tsung, “GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning,” in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 2390–2397.
- [21] H. Pang et al., “Asymptotic consistent graph structure learning for multivariate time-series anomaly detection,” *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–10, 2024, doi: [10.1109/TIM.2024.3369159](https://doi.org/10.1109/TIM.2024.3369159).
- [22] X. Zhou, C. Dai, W. Wang, and T. Qiu, “Global-local association discrepancy for multivariate time series anomaly detection in IIoT,” *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11287–11297, Apr. 2024, doi: [10.1109/JIOT.2023.3330696](https://doi.org/10.1109/JIOT.2023.3330696).
- [23] S. Qin, L. Chen, Y. Luo, and G. Tao, “Multiview graph contrastive learning for multivariate time-series anomaly detection in IoT,” *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22401–22414, Dec. 2023, doi: [10.1109/JIOT.2023.3303946](https://doi.org/10.1109/JIOT.2023.3303946).

- [24] S. He et al., "Graph structure learning-based multivariate time series anomaly detection in Internet of Things for human-centric consumer applications," *IEEE Trans. Consum. Electron.*, early access, Jun. 4, 2024, doi: [10.1109/TCE.2024.3409391](https://doi.org/10.1109/TCE.2024.3409391).
- [25] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [26] M. R. Khan and J. E. Blumenstock, "Multi-GCN: Graph convolutional networks for multi-view networks, with applications to global poverty," in *Proc. AAAI Conf. Artif. Intell.*, Jul. 2019, vol. 33, no. 1, pp. 606–613.
- [27] S. Fan, X. Wang, C. Shi, E. Lu, K. Lin, and B. Wang, "One2Multi graph autoencoder for multi-view graph clustering," in *Proc. Web Conf.*, Apr. 2020, pp. 3070–3076.
- [28] H. Xiao, J. Gao, D. S. Turaga, L. H. Vu, and A. Biem, "Temporal multi-view inconsistency detection for network traffic analysis," in *Proc. 24th Int. Conf. World Wide Web*, May 2015, pp. 455–465.
- [29] E. Gujral, R. Pasricha, and E. Papalexakis, "Beyond rank-1: Discovering rich community structure in multi-aspect graphs," in *Proc. Web Conf.*, Apr. 2020, pp. 452–462.
- [30] H. Yu et al., "Regularized graph structure learning with semantic knowledge for multi-variables time-series forecasting," 2022, *arXiv:2210.06126*.
- [31] Z. Zhong, L. Zheng, D. Cao, and S. Li, "Re-ranking person re-identification with K-reciprocal encoding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 1318–1327.
- [32] X. Zhang, M. Jiang, Z. Zheng, X. Tan, E. Ding, and Y. Yang, "Understanding image retrieval re-ranking: A graph neural network perspective," 2020, *arXiv:2012.07620*.
- [33] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 753–763.
- [34] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," 2017, *arXiv:1707.01926*.
- [35] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Jun. 2008, pp. 413–422.
- [36] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 3395–3404.
- [37] W. Liu et al., "Unsupervised deep anomaly detection for industrial multivariate time series data," *Appl. Sci.*, vol. 14, no. 2, p. 774, Jan. 2024.



Shiming He received the B.S. degree in information security and the Ph.D. degree in computer science and technology from Hunan University, China, in 2006 and 2013, respectively. She is currently an Associate Professor with the School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha, China. Her research interests include machine learning, data analysis, and anomaly detection.



Genxin Li received the B.S. degree from Jiangxi Science and Technology Normal University in 2021. He is currently pursuing the M.S. degree in computer science and technology with Changsha University of Science and Technology. His research interests include deep learning, data analysis, and anomaly detection.



Kun Xie (Member, IEEE) received the Ph.D. degree in computer application from Hunan University, Changsha, China, in 2007. She has published over 100 papers in major journals and conference proceedings, including journals such as *IEEE/ACM TRANSACTIONS ON NETWORKING*, *IEEE TRANSACTIONS ON MOBILE COMPUTING*, *IEEE TRANSACTIONS ON COMPUTERS*, *IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS*, *IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS*, and *IEEE TRANSACTIONS ON SERVICES COMPUTING*, and conferences, including *SIGMOD*, *INFOCOM*, *ICDCS*, *SECON*, *DSN*, and *IWQoS*. Her research interests include network measurement, network security, big data, and AI.



Pradip Kumar Sharma (Senior Member, IEEE) received the Ph.D. degree in CSE from Seoul National University of Science and Technology, South Korea, in August 2019.

He was a Post-Doctoral Research Fellow with the Department of Multimedia Engineering, Dongguk University, South Korea. He was a Software Engineer with MAQ Software, India, and was involved in a variety of projects, proficient in building largescale complex data warehouses, OLAP models, and reporting solutions that meet business objectives and align IT with business. He is currently an Assistant Professor of cybersecurity with the Department of Computing Science, University of Aberdeen, U.K. He has published many technical research articles in leading journals from IEEE, Elsevier, Springer, and MDPI. Some of his research findings are published in the most cited journals. His current research interests include cybersecurity, blockchain, edge computing, SDN, and the IoT security. He has also been invited to serve as a Technical Program Committee Member and the Chair in several reputed international conferences, such as *IEEE DASC 2021*, *IEEE CNCC 2021*, *CSA 2020*, *IEEE ICC 2019*, *IEEE MENACOMM 2019*, and *3ICT 2019*. He received a Top 1% Reviewer in computer science by Publons Peer Review Awards 2018 and 2019, Clarivate Analytics. He has been an expert reviewer for *IEEE TRANSACTIONS*, Elsevier, Springer, and MDPI journals and magazines. He is an Associate Editor of *Peer-to-Peer Networking and Applications (PPNA)*, *Human-Centric Computing and Information Sciences (HCIS)*, *Electronics (MDPI)*, and *Journal of Information Processing Systems (JIPS)*. He has been serving as the Guest Editor for international journals of certain publishers, such as IEEE, Elsevier, Springer, MDPI, and JIPS. He is listed in the World's Top 2% Scientists for citation impact during the calendar year 2019 by Stanford University.